

Intelligent Search Engine for Autocomplete & Fuzzy Query Processing

Project Proposal / Design Document

Project Overview

The objective of this project is to design and implement an intelligent search engine that provides efficient query retrieval through autocomplete, fuzzy search, and ranked query suggestions. The system combines advanced data structures with string matching algorithms while maintaining low query latency.

Objectives

- Develop an efficient search index using Trie.
- Support prefix-based autocomplete.
- Handle misspelled queries using Levenshtein distance.
- Rank suggestions using query frequency.
- Benchmark Trie-based indexing against linear search.

Proposed Architecture

Dataset → Index Construction → Trie + Frequency Map → Query Processor → Autocomplete / Spell Check / Ranking → Search Suggestions

Data Structures

Trie, Hash Map, Priority Queue (Heap), STL Vector

Algorithms

Trie insertion and prefix search, DFS-based autocomplete, Levenshtein distance, frequency-based Top-K ranking, linear search baseline.

Functionalities

Autocomplete, fuzzy search, ranked suggestions, dynamic insertion/deletion, benchmarking.

Dataset

A publicly available English dictionary containing approximately **370,000+ words**.

Technology Stack

Language: C++

Libraries: STL

Version Control: Git & GitHub

IDE: Visual Studio Code

Performance Metrics

Prefix lookup latency, autocomplete response time, spell correction latency, memory usage, and comparison with linear search.

Expected Outcome

Demonstrate efficient Trie-based indexing with autocomplete, fuzzy search, ranked suggestions, and improved query performance.

Future Scope

Compressed Trie, Inverted Index, TF-IDF ranking, GUI/Web interface, phrase search, and multi-threaded indexing.